Anjuman-I-Islam's

M. H. SABOO SIDDIK COLLEGE OF ENGINEERING

8, Saboo Siddik Polytechnic Road, Byculla, Mumbai, Maharashtra 400008

DEPARTMENT OF INFORMATION TECHNOLOGY

A Project Report

on

"CoreLLM: Building a Large Language Model Inference Engine"

Submitted in partial fulfilment of the requirements of the

ITP701 Major Project – I

Bachelor of Engineering in Information Technology (Semester VII)

Group No. 5

By

Harsh Desai (231416)
Himanshu Mishra (231417)
Rasil Khan (231462)
Kiran Jadhav (231419)

Under the guidance of

Er. Farzana Khan

REV - 2019 'C' Scheme

University of Mumbai
Academic Year (2026-27)

DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Students' Name, Roll Number and Signature

Harsh Desai	231416	……
Himanshu Mishra	231417	……
Rasil Khan	231462	……
Kiran Jadhav	231419	……

Date:

Place: Mumbai

CERTIFICATE

This is to certify that the project entitled “CoreLLM: Building a Large Language Model Inference Engine” is a bona fide work of “Harsh Desai” (231416), “Himanshu Mishra” (231417), “Rasil Khan” (231462) and “Kiran Jadhav” (231419) submitted to the University of Mumbai in partial fulfillment of the requirements for the ITP701 Major Project-I of 7th Semester in Department of Information Technology.

(Er. Farzana Khan)
Project Guide

(Dr. Zainab Mirza)
Head of Department

(Dr. Shafi Pathan)
Principal

PROJECT REPORT APPROVAL

This project report entitled “CoreLLM: Building a Large Language Model Inference Engine” is a bona fide work of “Harsh Desai” (231416), “Himanshu Mishra” (231417), “Rasil Khan” (231462) and “Kiran Jadhav” (231419), is approved for the ITP701 Major Project-I for the 7th semester.

Examiners

1) ----------

2) ----------

Date:

Place: Mumbai

ACKNOWLEDGEMENT

We, the project team, would like to express our sincere gratitude to everyone who contributed to this project. First and foremost, we thank our project guide, Er. Farzana Khan, for the guidance and encouragement provided throughout the work. We also thank our Major Project Coordinator, Er. Shrinidhi Gindi, for coordinating the Major Project-I activities.

We are grateful to Dr. Zainab Mirza, Head of the Department of Information Technology, and to the faculty and staff of Anjuman-I-Islam's M. H. Saboo Siddik College of Engineering for providing the academic environment and resources needed for this work.

Finally, we thank our families and classmates for their patience and support during the preparation of this report.

ABSTRACT

CoreLLM: Building a Large Language Model Inference Engine is a Major Project-I study of how a large language model (LLM) can be executed entirely on a local machine. Most applications that use LLMs send every request to a cloud API, which makes them depend on a working internet connection, exposes user data to a third party, and adds usage limits and costs. The technical foundation of this project is the research paper "A Foundational Approach to Local Large Language Model Execution", which describes a five-layer framework for local LLM execution: the hardware layer, the model layer, the inference engine layer, the augmentation layer, and the interface and orchestration layer.

For each layer the report explains the main constraint and the choice made in CodeHaven, an existing offline coding assistant for students that the research paper uses as its working example. CodeHaven runs a 4-bit quantized 7B coding model (qwen2.5-coder:7b) through Ollama on a laptop with 16 GB of RAM and no GPU, and adds local document retrieval with Chroma and on-device speech input with whisper.cpp. The report presents the memory needed at different quantization levels, the published benchmark scores of three 7B-class coding models, the way a request moves through the five layers, and the trade-offs between capability, privacy and hardware accessibility.

The report reproduces the evidence supplied in the source paper and does not add new experimental results. It concludes that local execution is not a full replacement for cloud LLMs, but that it fits well when privacy, offline use or cost matter and the task is well defined. Future work identified in the source includes routing requests between local and cloud models, testing lower-bit quantization and pruning for 8 GB machines, and building a small benchmark that compares local assistants across hardware tiers.

Keywords: Local Large Language Models, LLM Inference Engine, Quantization, GGUF, llama.cpp, Ollama, Retrieval-Augmented Generation, Offline AI.

TABLE OF CONTENTS

Sr. No.	Content	Page No.
	TITLE PAGE	i
	DECLARATION	ii
	CERTIFICATE	iii
	PROJECT REPORT APPROVAL	iv
	ACKNOWLEDGEMENT	v
	ABSTRACT	vi
	LIST OF FIGURES	viii
	LIST OF TABLES	viii
1	CHAPTER 1: INTRODUCTION 1.1 Project Overview; 1.2 Problem Statement; 1.3 Objectives and Scope	1
2	CHAPTER 2: LITERATURE SURVEY 2.1 Introduction; 2.2 Research Papers and Tools; 2.3 Summary of Related Work	4
3	CHAPTER 3: METHODOLOGY 3.1 Introduction; 3.2 System Design; 3.3 Team Roles and Contribution	8
4	CHAPTER 4: DESIGN / IMPLEMENTATION 4.1 Block Diagram; 4.2 System Architecture; 4.3 Memory Requirements, Quantization and Model Selection; 4.4 Data Flow: Retrieval-Augmented Pipeline; 4.5 System Workflow; 4.6 Module Description of the Working Example; 4.7 Privacy and Offline Design; 4.8 Status Reporting	10
5	CHAPTER 5: HARDWARE AND SOFTWARE REQUIREMENTS 5.1 Hardware Requirements; 5.2 Software Requirements	18
6	CHAPTER 6: TESTING / RESULT AND ANALYSIS 6.1 Results Reported in the Source; 6.2 Published Benchmark Scores; 6.3 Analysis; 6.4 Test Cases for the CoreLLM Project	19
7	CHAPTER 7: SNAPSHOTS 7.1 Main Window (Code Chat); 7.2 Notes & Q&A (RAG); 7.3 Exam Generator; 7.4 Generate Code; 7.5 Debug Assistant; 7.6 Local Storage & History	21
8	CHAPTER 8: TIMELINE CHART	27
9	CHAPTER 9: FUTURE SCOPE 9.1 Routing between Local and Cloud Models; 9.2 Lower-bit Quantization and Pruning; 9.3 Benchmark across Hardware Tiers; 9.4 Larger Models for Better Hardware; 9.5 Terminal-Based Local Inference Engine (Lumen Direction)	28
10	CHAPTER 10: CONCLUSION 10.1 Achievement of Objectives; 10.2 Key Learnings; 10.3 Limitations and Trade-Offs; 10.4 Social and Educational Impact; 10.5 Final Remarks	29
11	REFERENCES	31

LIST OF FIGURES

Sr. No.	Title	Page No.
1	Figure 1: Five-layer framework for local LLM execution	10
2	Figure 2: Approximate model file size at common quantization levels (values from Table 4.1)	13
3	Figure 3: Published pass@1 scores of three 7B-class instruction-tuned coding models on HumanEval, MBPP and their EvalPlus versions	14
4	Figure 4: Path of a voice question through the layers	15
5	Figure 5: CodeHaven main window	21
6	Figure 6: Notes & Q&A (RAG) screen of CodeHaven	22
7	Figure 7: Exam Generator screen of CodeHaven	23
8	Figure 8: Generate Code screen of CodeHaven	24
9	Figure 9: Debug Assistant screen of CodeHaven	24
10	Figure 10: Local Storage & History panel, Chats tab	25
11	Figure 11: Local Storage & History panel, Notes Q&A tab	25
12	Figure 12: Local Storage & History panel, Exam Papers tab	26
13	Figure 13: Proposed 12-week project timeline (planned, not completed work)	27

LIST OF TABLES

Sr. No.	Title	Page No.
1	Table 2.1: Summary of related work and its role in CodeHaven (Table I of the research paper)	6
2	Table 3.1: Constraint and choice at each layer of the framework	8
3	Table 4.1: Approximate model file size by quantization level (Table II of the research paper)	12
4	Table 4.2: Stages of the local retrieval-augmented generation pipeline	14
5	Table 4.3: Steps of the request and the layer involved	15
6	Table 6.1: Observations reported in the research paper	19
7	Table 6.2: Published pass@1 scores (%) of three 7B-class instruction-tuned coding models [10]	19

CHAPTER 1
INTRODUCTION

1.1 Project Overview

Large language models (LLMs) based on the transformer [1] are now part of everyday software such as chat assistants, code completion tools and document search. Open-weight models like LLaMA [2] can be downloaded and run by anyone. Even so, most consumer applications still send every request to a cloud API, even when the model behind it is openly available.

CoreLLM: Building a Large Language Model Inference Engine is the Major Project-I of Group 5 of the Department of Information Technology. The project studies and documents how an LLM inference engine fits into a complete local system. It is based on the research paper "A Foundational Approach to Local Large Language Model Execution", which organises local LLM execution into five layers – hardware, model, inference engine, augmentation, and interface and orchestration – and states for each layer the main constraint and the choice that was made.

The research paper uses **CodeHaven**, an offline coding assistant for students, as its working example. CodeHaven is an existing application: it runs a 4-bit quantized 7B model (qwen2.5-coder:7b) through Ollama [9] on a laptop with 16 GB of RAM and no GPU, adds local document retrieval, and accepts on-device speech input. In this report CodeHaven is used only to illustrate the framework and its screenshots are shown as evidence of the working example; it is not presented as a separate CoreLLM application.

Running the model on the user's own laptop became practical only recently, because two things arrived together: good open-weight models with 1 to 13 billion parameters, and post-training quantization methods such as GPTQ [4] and AWQ [5], which shrink a model to about a quarter of its original size with only a small drop in quality. With an efficient inference engine, a quantized 7B model runs interactively on a laptop with 8 to 16 GB of RAM.

1.2 Problem Statement

1.2.1 Problem Definition

Applications that depend on a cloud LLM API create everyday problems, particularly for students:

- **Connectivity barrier:** the tool stops working when the internet connection drops.
- **Usage limits:** free tiers have rate limits that tend to run out in the middle of a lab session or an assignment.
- **Privacy risk:** every prompt, including the code or notes pasted into it, goes to someone else's server.

Xu et al. [3] report that latency, the risk of sending data out and recurring cost are the main reasons users prefer on-device models. Therefore, there is a need to understand how an LLM can be run entirely on ordinary laptop hardware, and which choices at each level of the system make this possible.

1.2.2 Proposed Work

The project proposes to document and explain a five-layer framework for local LLM execution, from the hardware up to the user interface, with the inference engine at its centre. The report summarizes the related work behind each layer, explains the memory limits that make quantization a requirement, compares model choices using published benchmark scores, and follows a request through all five layers using CodeHaven as the working example.

1.2.3 Proposed Methodology

The system is organised bottom-up. The authors observed that "every choice near the bottom of the stack limits what is possible above it", so the hardware limit is treated as the input to every other design decision. The methodology therefore moves from the hardware layer (memory limit) to the model layer (quantized model), the inference engine layer (Ollama on llama.cpp), the augmentation layer (local retrieval) and finally the interface and orchestration layer (desktop application, routing, storage and speech input). Details of any CoreLLM-specific engine implementation beyond the source paper are **[to be added by the project team]**.

1.3 Objectives and Scope

1.3.1 Objectives

The main objectives of the project are:

- To explain the five-layer framework for running an LLM entirely on a local machine.

- To show why memory is the main limit on a laptop and why quantization is a requirement and not an optional speed-up.
- To describe the model layer, including GGUF quantization levels and the file sizes they produce.
- To explain the role of the inference engine (Ollama on llama.cpp), the key-value cache and the context length.
- To explain how retrieval-augmented generation lets a quantized model answer questions about material it has never seen, without any data leaving the device.
- To describe the interface and orchestration layer, including local history storage and on-device speech input.
- To discuss the trade-offs between capability and privacy, hardware accessibility and hybrid deployment.

1.3.2 Scope

The scope of this report covers the following, all taken from the source paper:

- The five layers of local LLM execution and the choices made at each layer in CodeHaven.
- Approximate memory requirements at 3B, 7B and 13B parameter sizes and at 4-bit, 8-bit and 16-bit precision.
- A comparison of CodeLlama-7B-Instruct, DeepSeek-Coder-6.7B-Instruct and Qwen2.5-Coder-7B-Instruct using the published scores reported in the Qwen2.5-Coder technical report [10].
- The CodeHaven working example: Code Chat, Notes & Q&A, Exam Generator, Generate Code and Debug Assistant modes on Windows laptops for Python and JavaScript students.

The project does not claim new experimental findings. Latency measurements on student laptops, lower-bit quantization, pruning and local/cloud routing are identified in the source as future work (Chapter 9).

CHAPTER 2
LITERATURE SURVEY

2.1 Introduction

The literature survey follows the related work section of the research paper and its ten references. These works cover the transformer and open-weight models, model compression and quantization, on-device inference, retrieval-augmented generation, and local speech recognition. Together they provide the foundation for each layer of the framework (Table 2.1 and Figure 1).

The survey shows that quantization and efficient inference engines now make it possible to run 1–13 billion parameter models on laptop-class hardware, but that the existing works address individual pieces. The on-device survey of Xu et al. [3] explains why local execution is worth doing, but it does not describe a complete application that an end user can install. The framework used in this project starts from this point and treats the hardware limit as the input to every other design decision.

2.2 Research Papers and Tools

Paper 1: Attention Is All You Need
Authors: A. Vaswani et al.
Published: 2017 (NeurIPS)
The transformer architecture on which present-day large language models are based [1]. The research paper cites it as the starting point of the LLMs that now appear in chat assistants, code completion tools and document search.

Paper 2: LLaMA: Open and Efficient Foundation Language Models
Authors: H. Touvron et al.
Published: 2023
An example of open-weight models that can be downloaded and run by anyone [2]. It illustrates that the model behind an application can be openly available even when the application still calls a cloud API.

Paper 3: On-Device Language Models: A Comprehensive Review
Authors: J. Xu et al.
Published: 2024
A survey of on-device language models covering efficient architectures, compression, hardware acceleration and edge–cloud collaboration [3]. It reports that latency, the risk of

sending data out and recurring cost are the main reasons users prefer on-device models. It is the motivation for the layered design.

Paper 4: GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers
Authors: E. Frantar, S. Ashkboos, T. Hoefler and D. Alistarh
Published: 2023 (ICLR)
GPTQ quantizes one layer at a time and uses approximate second-order information to keep the output error small, so that large models can be reduced to 3 or 4 bits per weight with little loss in accuracy [4]. It explains why a 7B model fits in laptop RAM. Limitation: it needs a calibration dataset.

Paper 5: AWQ: Activation-aware Weight Quantization for On-device LLM Compression and Acceleration
Authors: J. Lin et al.
Published: 2024 (MLSys)
AWQ starts from the observation that only a small share of weight channels matters most to the output. It uses activation statistics to find these channels and scales them so that they survive quantization [5]. It is a second route to 4-bit models and also needs calibration data.

Paper 6: llama.cpp: LLM Inference in C/C++
Authors: G. Gerganov
Published: 2023 (GitHub repository)
Runs quantized models in plain C/C++ on a CPU, with optional GPU offload [6]. The GGUF format it uses stores quantized weights together with the metadata needed to run them quickly on a CPU. It is the engine underneath Ollama. Limitation: it is a command-line tool with no application features.

Paper 7: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
Authors: P. Lewis et al.
Published: 2020 (NeurIPS)
Retrieval-augmented generation (RAG) fetches relevant passages from an external collection at query time and gives them to the model along with the question [7]. The original work used a large Wikipedia index. The same idea works at a much smaller scale as a local vector store over a student's own notes.

Paper 8: Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)
Authors: A. Radford et al.
Published: 2022
Whisper was trained on a large amount of weakly labelled multilingual audio and transcribes speech accurately [8]. Its C/C++ port, whisper.cpp, runs on a CPU, so it can work offline next to a local LLM. Limitation: it is heavy for laptops if run as-is.

Paper 9: Ollama: Get up and running with large language models locally
Authors: Ollama Inc.
Published: 2024
Downloads, manages and serves local models through a simple API [9]. It is built on llama.cpp and is the inference runtime used in the working example. It has no document search, voice or user interface of its own.

Paper 10: Qwen2.5-Coder Technical Report
Authors: B. Hui et al.
Published: 2024
The report from which the published HumanEval and MBPP pass@1 scores in Figure 3 are taken [10]. The 7B instruction-tuned variant is the model used in CodeHaven.

2.3 Summary of Related Work

Table 2.1 reproduces the summary table of the research paper. For each work it gives what the work does, its main limitation and the role it plays in the working example.

Table 2.1: Summary of related work and its role in CodeHaven (Table I of the research paper)

Work	What it does	Limitation	Role in CodeHaven
GPTQ [4]	Shrinks a trained model to 3–4 bits per weight, one layer at a time	Needs a calibration dataset	Explains why a 7B model fits in laptop RAM
AWQ [5]	Protects the few weight channels that matter most, using activation statistics	Also needs calibration data	Second route to 4-bit models
llama.cpp [6]	Runs quantized models in plain C/C++ on a CPU, with optional GPU offload	Command-line tool with no app features	Engine underneath Ollama
Ollama [9]	Downloads, manages and serves local models through a simple API	No document search, voice or UI	Our inference runtime (qwen2.5-coder:7b)
Xu et al. [3]	Surveys on-device LLMs: compression, hardware, edge–cloud setups	A survey, not a working end-user tool	Motivation for the layered design

Work	What it does	Limitation	Role in CodeHaven
RAG [7]	Retrieves passages and gives them to the model with the question	Built around a large Wikipedia index	Small local version over students' own notes
Whisper [8]	Multilingual speech recognition trained on large web audio	Heavy for laptops if run as-is	Offline voice input (via whisper.cpp)
CodeHaven (ours)	Chat, code generation, debugging, notes Q&A and voice in one offline desktop app	Limited by a 7B model on student laptops	—

The reviewed work demonstrates that quantization can compress models to about 4 bits with little loss in quality, that llama.cpp and Ollama can run them on a CPU, that retrieval can ground answers in local documents and that speech recognition can also run on the device. What is missing in the existing work is a description of how these pieces fit together in an application that an end user can install. The five-layer framework fills this gap.

CHAPTER 3
METHODOLOGY

3.1 Introduction

The methodology of the project follows the structure of the research paper. When the authors started building CodeHaven they expected that running a model locally would simply mean replacing a cloud API call with a local one. In practice, every choice near the bottom of the stack limits what is possible above it. The system was therefore organized into five layers and described from the bottom up, stating at each layer what was used and why.

3.2 System Design

The framework consists of the following five layers, from the bottom of the stack to the top. The choice made in the working example at each layer is shown in the last column of Table 3.1.

Table 3.1: Constraint and choice at each layer of the framework

No.	Layer	Main constraint	Choice in CodeHaven
1	Hardware	Memory (RAM) is the main limit on a laptop	Intel Core i7, 16 GB RAM, no discrete GPU, Windows 11
2	Model	A 16-bit 7B model needs about 14 GB for its weights alone	qwen2.5-coder:7b, 4-bit GGUF (about 4.7 GB on disk)
3	Inference Engine	Context length trades off against free memory through the key-value cache	Ollama on llama.cpp with a local HTTP API
4	Augmentation	A quantized model only knows what it saw during training	Chunks, all-MiniLM-L6-v2 embeddings and Chroma (local RAG)
5	Interface & Orchestration	Everything must run without a network or cloud account	Electron app, FastAPI routing, SQLite history, whisper.cpp voice input

3.2.1 System Planning

Planning followed from the problems identified in Section 1.2.1:

1. Identify what bothers students most about cloud-based AI coding assistants. The authors asked fellow students; three answers kept coming up: the tools need the internet all the time, the free tiers hit rate limits, and people are not comfortable sending their own code to an outside server.
2. Treat these answers as the main requirements: offline operation, no usage limits and data that stays on the device.
3. Fix the hardware limit first and treat quantization as a requirement.

4. Select the model and the inference engine, then add retrieval and the interface layer on top.

3.2.2 Layered Design Approach

The design approach is layered and bottom-up. Each layer is chosen after the one below it because the lower layers limit what is possible above. The approach consists of the following stages:

1. **Hardware:** record the processor, RAM, GPU and operating system of the target laptop.
2. **Model:** choose a model size and quantization level that fits in the available memory.
3. **Inference engine:** run the model with an engine that works on the CPU alone and manages the key-value cache; set the context length.
4. **Augmentation:** add local retrieval for questions about the user's own material.
5. **Interface and orchestration:** build the desktop application that routes each request, stores the history and accepts speech.

The approach keeps every layer replaceable with tools that already exist; the authors observe that every layer of the framework could be filled with tools that already exist.

3.3 Team Roles and Contribution

Team Member	Roll No	Role	Key Contributions	Contribution
Harsh Desai	231416	Documentation and Presentation Lead	Report preparation, documentation, research support, review, and presentations.	23%
Himanshu Mishra	231417	Project Ideation and Technical Lead	Proposed the project idea, leads technical implementation, and will lead future development.	27%
Rasil Khan	231462	Research and Presentation Lead	Research and technical analysis support, documentation support, and presentations.	27%
Kiran Jadhav	231419	Research and Coordination Support	Research assistance, coordination, reminders, and review support.	23%
Total				**100%**

CHAPTER 4
DESIGN / IMPLEMENTATION

This chapter describes the five-layer framework and the CodeHaven working example. Everything described is taken from the research paper and from the CodeHaven screenshots supplied with it.

4.1 Block Diagram

Figure 1 is the block diagram of the framework. It shows the five layers as a stack. Each box states what CodeHaven uses at that layer and the related work from Chapter 2 that it builds on.

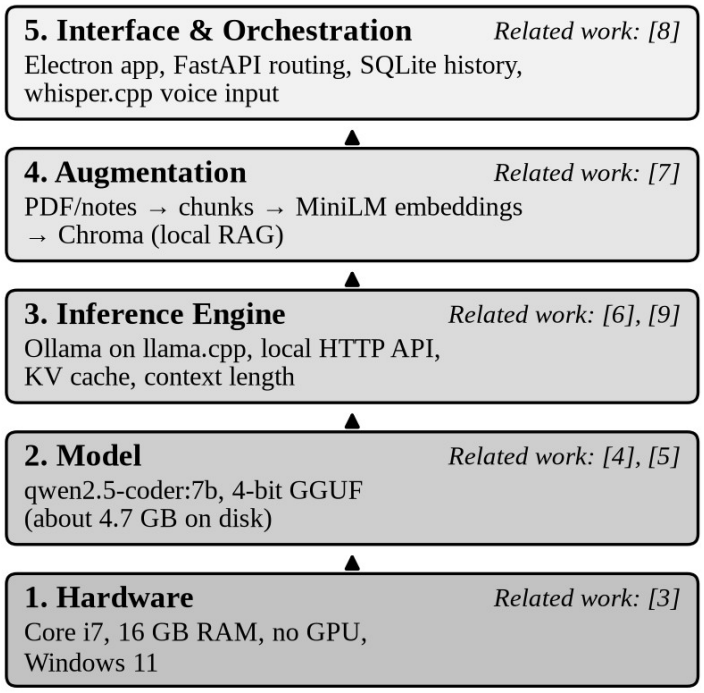


Figure 1: Five-layer framework for local LLM execution (Fig. 1 of the research paper)

4.1.1 Detailed explanation of Figure 1

Figure 1 reads from the bottom to the top. The arrows between the boxes show that each layer rests on the one below it, so the choices made at the bottom limit everything above.

1. Hardware (related work [3]). Core i7, 16 GB RAM, no GPU, Windows 11. The on-device survey of Xu et al. motivates treating the hardware as the first design decision.

2. Model (related work [4], [5]). qwen2.5-coder:7b in a 4-bit GGUF file of about 4.7 GB on disk. GPTQ and AWQ are the quantization methods that justify the 4-bit size.

3. Inference Engine (related work [6], [9]). Ollama on llama.cpp, a local HTTP API, the key-value cache and the context length.

4. Augmentation (related work [7]). PDF or notes are split into chunks, converted into MiniLM embeddings and stored in Chroma, which is the local RAG store.

5. Interface & Orchestration (related work [8]). An Electron app, FastAPI routing, SQLite history and whisper.cpp voice input.

4.2 System Architecture

4.2.1 Layer 1 – Hardware Layer

Everything else depends on the machine. CodeHaven was built and tested on a laptop with an Intel Core i7 processor, 16 GB of RAM, no discrete GPU and Windows 11. Memory was the main limit. A 7B-parameter model stored in 16-bit precision needs about 14 GB for its weights alone (7 billion parameters at 2 bytes each), as expressed in Equation (1):

$$\text{Memory} \approx \text{parameters} \times \text{bytes per parameter} = 7\times10^{9} \times 2 \approx 14 \text{ GB} \qquad (1)$$

On a 16 GB laptop that leaves almost nothing for the operating system, the desktop app and the document index, and on an 8 GB laptop the model cannot be loaded at all. For this reason quantization is treated as a requirement, not as an optional speed-up.

4.2.2 Layer 2 – Model Layer

CodeHaven uses qwen2.5-coder:7b at 4-bit quantization, stored in the GGUF format used by llama.cpp [6]. GPTQ [4] and AWQ [5] showed that weights can be compressed to around 4 bits with little loss in output quality, and the 4-bit GGUF formats follow the same idea. Table 4.1 lists typical file sizes at common quantization levels and Figure 2 plots them against 8 GB and 16 GB of laptop RAM. The choice of the model itself is discussed in Section 4.3.

4.2.3 Layer 3 – Inference Engine Layer

Ollama [9] loads and runs the model. It is built on llama.cpp [6], so it works on the CPU alone, can offload layers to a GPU when one is present, and manages the key-value cache during a conversation. Ollama exposes a local HTTP API, and the FastAPI backend sends all model requests to it. The main setting at this layer is the context length. A longer context

means a larger cache and more RAM, so conversation length trades off directly against free memory.

4.2.4 Layer 4 – Augmentation Layer

A quantized model only knows what it saw during training, so for questions about a student's own material retrieval is added. In the Notes & Q&A mode, a user uploads PDF, DOCX, TXT, Markdown or code files. CodeHaven splits each file into chunks (a 238 KB PDF in the working example's library became 16 chunks), converts the chunks into embeddings with the all-MiniLM-L6-v2 model, and stores them in Chroma. When the user asks a question, the closest chunks are retrieved and placed in the prompt, following the retrieval-augmented generation approach of Lewis et al. [7]. The answer is shown with citations to the source notes. The documents and the index stay on the device, so this layer adds no network dependency. The stages are summarized in Table 4.2.

4.2.5 Layer 5 – Interface and Orchestration Layer

The top layer is an Electron desktop application (Figure 4). It has five modes: Code Chat, Notes & Q&A, Exam Generator, Generate Code and Debug Assistant, with a code editor beside the chat panel. The FastAPI backend routes each request to the right module. A debug request, for example, sends the code from the editor together with the pasted error message, while a notes question goes through retrieval first. Chats, note sessions and generated exam papers are saved in a local SQLite database (codehaven.db), so the history is still there after a restart and no cloud account is needed. A microphone button next to the Send button records speech, which the Whisper base model, run through whisper.cpp, transcribes on the device. The text then appears in the input box. Status badges at the top of the window show whether the API and Ollama are running, which model is loaded, and that the app is in offline mode.

4.3 Memory Requirements, Quantization and Model Selection

Table 4.1: Approximate model file size by quantization level (Table II of the research paper)

Params	Quantization	Size	Typical use
3B	4-bit (Q4_K_M)	~2 GB	Quick autocomplete and short chat
7B	4-bit (Q4_K_M)	~4–5 GB	Coding help and notes Q&A (CodeHaven default)
7B	8-bit (Q8_0)	~7–8 GB	Slightly better output if RAM allows
13B	4-bit (Q4_K_M)	~7–8 GB	Better generation, needs a 16 GB machine
7B	16-bit (F16)	~14 GB	Too large for most laptops

Rounded values. Exact sizes depend on the model architecture and the quantization scheme [4]–[6].

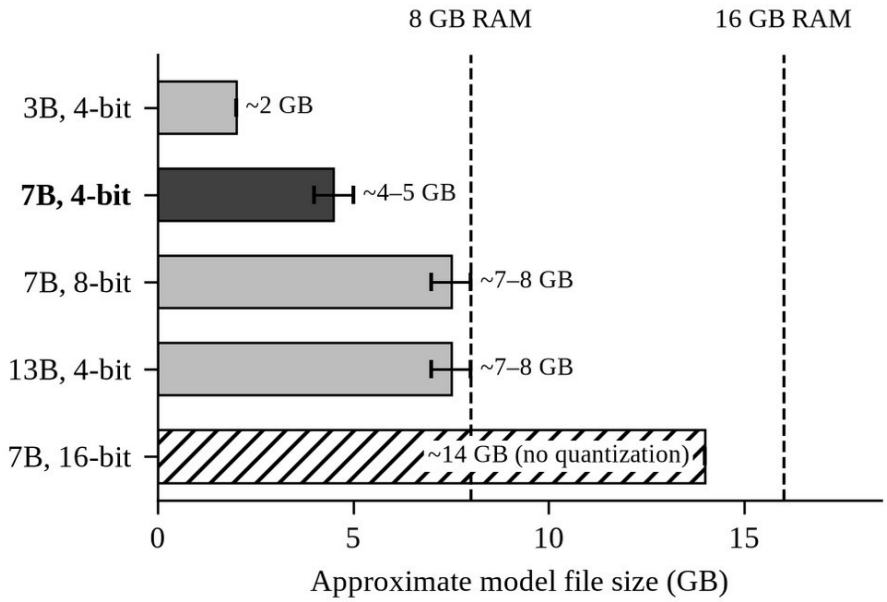


Figure 2: Approximate model file size at common quantization levels (values from Table 4.1). Error bars show the size range. Dashed lines mark 8 GB and 16 GB of laptop RAM. The dark bar is the setting used in CodeHaven. (Fig. 2 of the research paper)

A 3B model at 4-bit needs about 2 GB, and a 7B model at 4-bit needs about 4–5 GB, so both fit in 8 GB of RAM, although the 7B model leaves little room once the operating system and the app are running. The 8-bit 7B model and the 4-bit 13B model both need about 7–8 GB for the weights alone, which in practice means a 16 GB machine, and the unquantized 7B model does not fit comfortably in either. These are file sizes. The running model also needs memory for its key-value cache, which grows with the context length, so real usage is higher. The 4-bit 7B model was chosen as the default because it gave the best quality that still left headroom on a student laptop.

4.3.1 Model selection

To pick the model itself, the authors tried CodeLlama, DeepSeek-Coder and Qwen2.5-Coder at the 7B scale on their own coding and debugging prompts. Qwen2.5-Coder gave more accurate code completions and followed instructions better than the other two. The published scores in Figure 3 agree with this: on HumanEval and MBPP, and on their stricter EvalPlus versions, Qwen2.5-Coder-7B-Instruct scores highest, while CodeLlama-7B-Instruct is far behind [10].

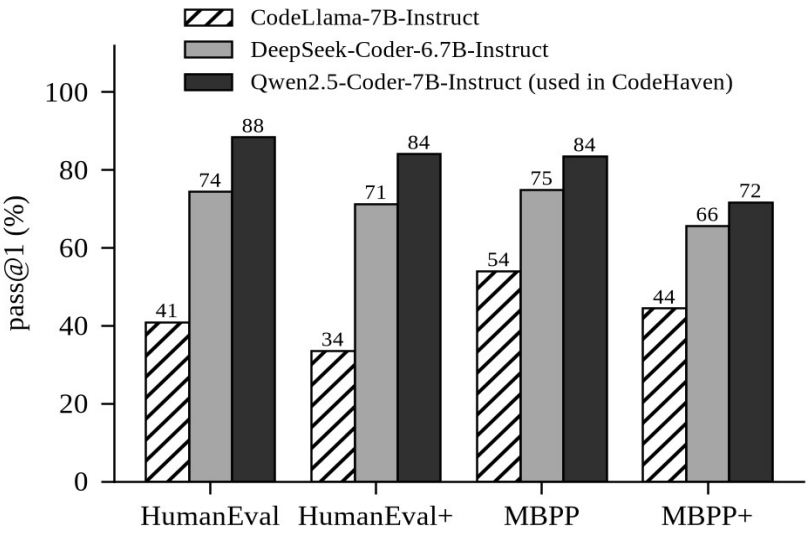


Figure 3: Published pass@1 scores of three 7B-class instruction-tuned coding models on HumanEval, MBPP and their EvalPlus versions (HumanEval+, MBPP+). Data from the Qwen2.5-Coder technical report [10]. (Fig. 3 of the research paper)

4.4 Data Flow: Retrieval-Augmented Pipeline

The augmentation layer processes a student's notes in the stages of Table 4.2. All stages run on the device.

Table 4.2: Stages of the local retrieval-augmented generation pipeline

Stage	Description (from the research paper)	Where it runs
1. Upload	The user uploads PDF, DOCX, TXT, Markdown or code files in Notes & Q&A mode	Electron interface
2. Chunk	Each file is split into chunks (a 238 KB PDF became 16 chunks)	On the device
3. Embed	The chunks are converted into embeddings with the all-MiniLM-L6-v2 model	On the device
4. Store	The embeddings are stored in Chroma, a local vector store	On the device
5. Retrieve	When a question is asked, the closest chunks are retrieved	On the device
6. Generate	The chunks are placed in the prompt and qwen2.5-coder:7b answers through Ollama; the answer is shown with citations to the source notes	Ollama, on the CPU

4.5 System Workflow

The research paper follows one request through the layers. A student uploads a lecture PDF in Notes & Q&A mode and asks, by voice, how merge sort works. Figure 4 shows the path of this request and Table 4.3 explains each step. No step in this chain uses the network.

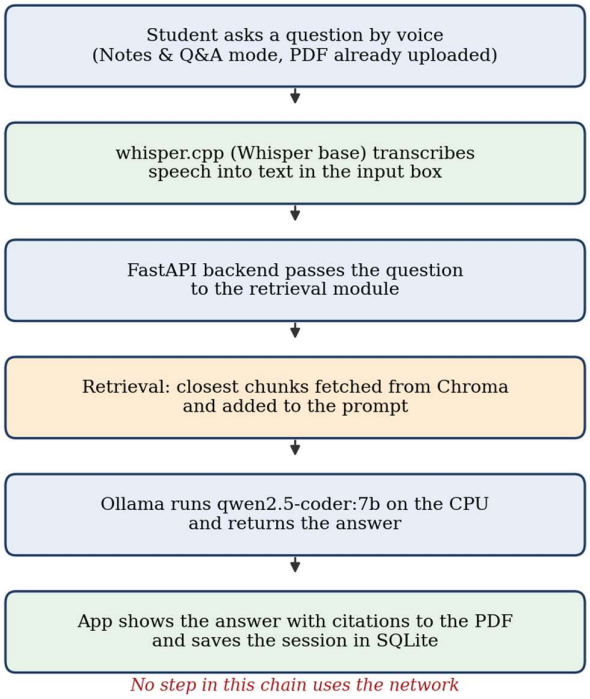


Figure 4: Path of a voice question through the layers (redrawn from the example in Section IV-B of the research paper)

Table 4.3: Steps of the request and the layer involved

Step	What happens	Layer	Component
1	The student speaks the question	Interface & Orchestration	Microphone button
2	Speech is transcribed into text in the input box	Interface & Orchestration	Whisper base model through whisper.cpp
3	The question is passed to the retrieval module	Interface & Orchestration	FastAPI backend
4	The closest chunks are fetched and added to the prompt	Augmentation	Chroma
5	The model produces the answer on the CPU	Inference Engine / Model	Ollama running qwen2.5-coder:7b
6	The answer is shown with citations to the PDF and saved	Interface & Orchestration	Electron app, SQLite

4.6 Module Description of the Working Example

CodeHaven has five modes (Figure 5 in Chapter 7 shows the main window). The descriptions below are limited to what the research paper and the screenshots state.

4.6.1 Code Chat

A chat panel with a code editor beside it, working with the local model. The welcome message in the main window invites the student to ask about Python, JavaScript, React or Node.js and to paste code in the editor.

4.6.2 Generate Code

The student describes the code to be built, picks a language (Python is selected in the screenshot) and clicks Generate. The code appears in the editor automatically.

4.6.3 Debug Assistant

The student pastes code into the editor and an optional error message or traceback below the chat. The application sends the code from the editor together with the pasted error message, explains what is wrong and suggests a fix; an Apply fix action replaces the editor content.

4.6.4 Notes & Q&A (RAG)

The student uploads notes and asks questions. The request goes through retrieval first (Section 4.2.4) and the answer is shown with citations. The question can be scoped to all uploaded notes or to one note.

4.6.5 Exam Generator

The Exam Settings panel takes the subject or course title, paper title, target notes, difficulty level (Easy, University, Hard), total marks and duration, and an option to include model answers and a step-by-step marking scheme. Generated papers are saved in the local database.

4.7 Privacy and Offline Design

The working example is designed so that no step uses the network. The documents and the index stay on the device, the history is stored in a local SQLite database (codehaven.db) with no cloud account, speech is transcribed on the device by whisper.cpp, and the status badges show that the application is in offline mode. The history panel of the application itself states “100% Offline & Private”.

4.8 Status Reporting

Status badges at the top of the window show whether the API and Ollama are running, which model is loaded, and that the application is in offline mode. They make the state of the lower layers visible to the student before a prompt is sent.

CHAPTER 5
HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements

The following hardware is the machine on which the working example was built and tested, as reported in the research paper:

Component	Requirement / Configuration
Processor	Intel Core i7
RAM	16 GB (a 4-bit 7B model fits in 8 GB but leaves little room; a 16-bit 7B model needs about 14 GB)
Graphics	No discrete GPU; a GPU is optional and allows Ollama to offload layers
Storage	About 4.7 GB on disk for the 4-bit qwen2.5-coder:7b model, plus the document index
Operating system	Windows 11
Microphone	Needed for the voice-input button

5.2 Software Requirements

The following software components are named in the research paper:

Software	Purpose
Ollama	Local LLM runtime that loads and serves the model through a local HTTP API
llama.cpp	Engine underneath Ollama; runs quantized GGUF models on the CPU
qwen2.5-coder:7b (4-bit GGUF)	Code language model
all-MiniLM-L6-v2	Embedding model for the retrieval pipeline
Chroma	Local vector store for document chunks
Whisper base model with whisper.cpp	On-device speech recognition
Electron	Desktop application shell
FastAPI	Backend that routes each request to the right module
SQLite (codehaven.db)	Local storage of chats, note sessions and exam papers

Versions of Python, Node.js and other development tools are not stated in the source and are **[to be added by the team]** if required.

CHAPTER 6
TESTING / RESULT AND ANALYSIS

The research paper reports measurements and observations from building the working example; it does not publish a test-case table or latency measurements. This chapter therefore presents only the results that the source supplies. No test results have been added.

6.1 Results Reported in the Source

Table 6.1: Observations reported in the research paper

Item	Observation
Configuration	4-bit quantized 7B model (qwen2.5-coder:7b) run through Ollama on a laptop with 16 GB RAM and no GPU
Model file	About 4.7 GB on disk (4-bit GGUF)
Memory of a 16-bit 7B model	About 14 GB for the weights alone
Retrieval example	A 238 KB PDF became 16 chunks in the notes library
Model selection	On the authors' own coding and debugging prompts, Qwen2.5-Coder gave more accurate code completions and followed instructions better than CodeLlama and DeepSeek-Coder
Offline operation	No step of the example request (voice, retrieval, generation, storage) uses the network

6.2 Published Benchmark Scores

Table 6.2 gives the pass@1 values shown in Figure 3. They are published scores from the Qwen2.5-Coder technical report [10] and were not measured by the project team.

Table 6.2: Published pass@1 scores (%) of three 7B-class instruction-tuned coding models [10]

Model	HumanEval	HumanEval+	MBPP	MBPP+
CodeLlama-7B-Instruct	41	34	54	44
DeepSeek-Coder-6.7B-Instruct	74	71	75	66
Qwen2.5-Coder-7B-Instruct (used in CodeHaven)	88	84	84	72

6.3 Analysis

Memory is the deciding result. Because a 16-bit 7B model needs about 14 GB for its weights, quantization is what allows the model to run on student laptops; the 4-bit 7B model, at about 4–5 GB, leaves headroom on a 16 GB machine. The key-value cache grows with the context length, so real memory use is higher than the file size and conversation length trades off against free memory. On the benchmarks of Table 6.2, Qwen2.5-Coder-

7B-Instruct scores highest on all four measures, which agrees with the authors' own experience and supports its use as the default model.

The trade-offs found while building the working example are discussed in Section 10.3.

6.4 Test Cases for the CoreLLM Project

The table below is the structure for the project's own test cases. The source documents give no test inputs or outcomes, so the entries are left blank and must be completed when testing is carried out.

ID	Test Case	Input	Expected Output	Actual Output	Status
TC01	Local Model Loading	Valid path to a supported local model file	Model loads and becomes available for inference	To be recorded during testing	Pending
TC02	Prompt Processing	"Explain machine learning in simple terms."	System accepts the prompt and generates a relevant response	To be recorded during testing	Pending
TC03	Empty Prompt Handling	Empty text input	System rejects the empty prompt or requests valid input without crashing	To be recorded during testing	Pending
TC04	Invalid Model Path	Path to a non-existent model file	System displays an appropriate error and handles the failure gracefully	To be recorded during testing	Pending

CHAPTER 7
SNAPSHOTS

The following figures show the **CodeHaven** desktop application, which is the working example of the research paper. They are screenshots of CodeHaven and not of a separate CoreLLM application. The status badges in every screenshot show API online, Ollama ready, the qwen2.5-coder:7b model and Offline mode.

7.1 Main Window (Code Chat)

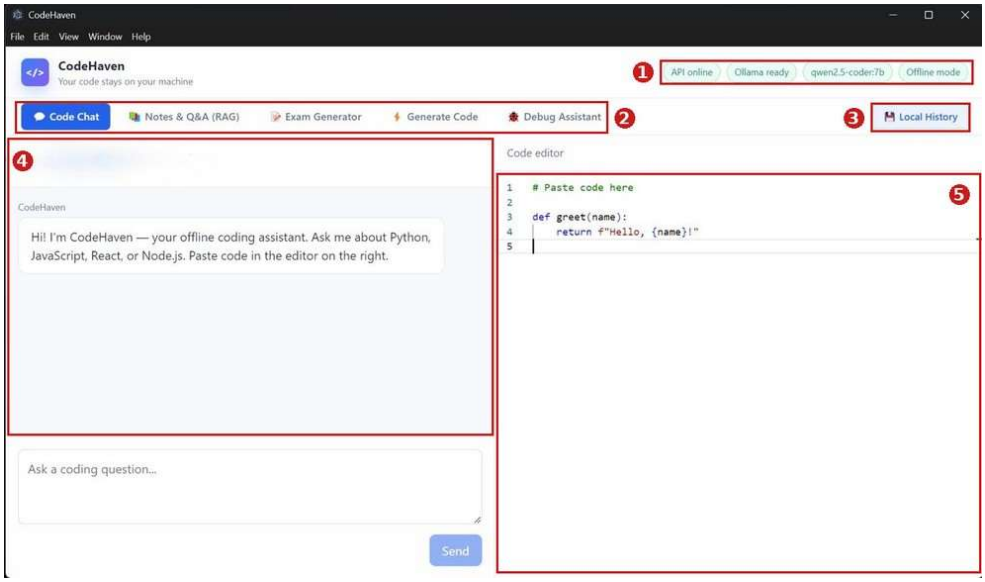


Figure 5: CodeHaven main window: (1) status badges for the API, Ollama, the loaded model and offline mode; (2) the five modes; (3) locally stored history; (4) chat panel; (5) code editor (Fig. 4 of the research paper)

7.1.1 Explanation of Figure 5

The numbered regions are the ones marked in the paper. Region 1 shows the status badges that report the state of the API, Ollama, the loaded model and offline mode. Region 2 holds the five modes: Code Chat, Notes & Q&A (RAG), Exam Generator, Generate Code and Debug Assistant. Region 3 is the Local History button for locally stored chats, notes and exams. Region 4 is the chat panel, which carries the welcome message and the “Ask a coding question…” box with the Send button, and region 5 is the code editor.

7.2 Notes & Q&A (RAG)

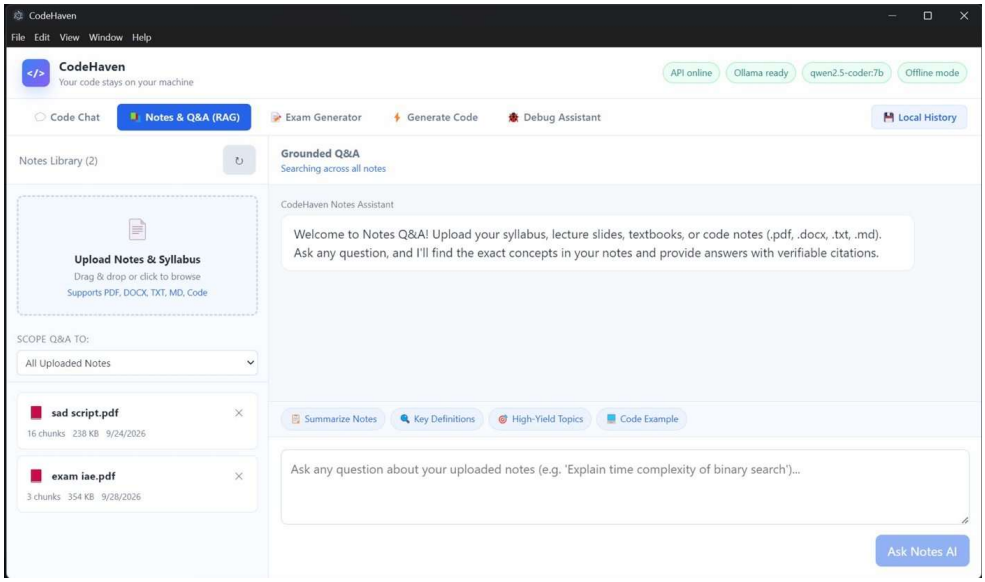


Figure 6: Notes & Q&A (RAG) screen of CodeHaven

7.2.1 Explanation of Figure 6

The Notes Library on the left lists two uploaded notes with their number of chunks, size and date: “sad script.pdf” (16 chunks, 238 KB) and “exam iae.pdf” (3 chunks, 354 KB). The first entry matches the example in the research paper of a 238 KB PDF that became 16 chunks. An upload area accepts PDF, DOCX, TXT, MD and code files, and a “Scope Q&A to” list lets the student choose all uploaded notes or a single note. The right side is titled “Grounded Q&A”; quick prompts (Summarize Notes, Key Definitions, High-Yield Topics, Code Example) sit above the question box and the Ask Notes AI button.

7.3 Exam Generator

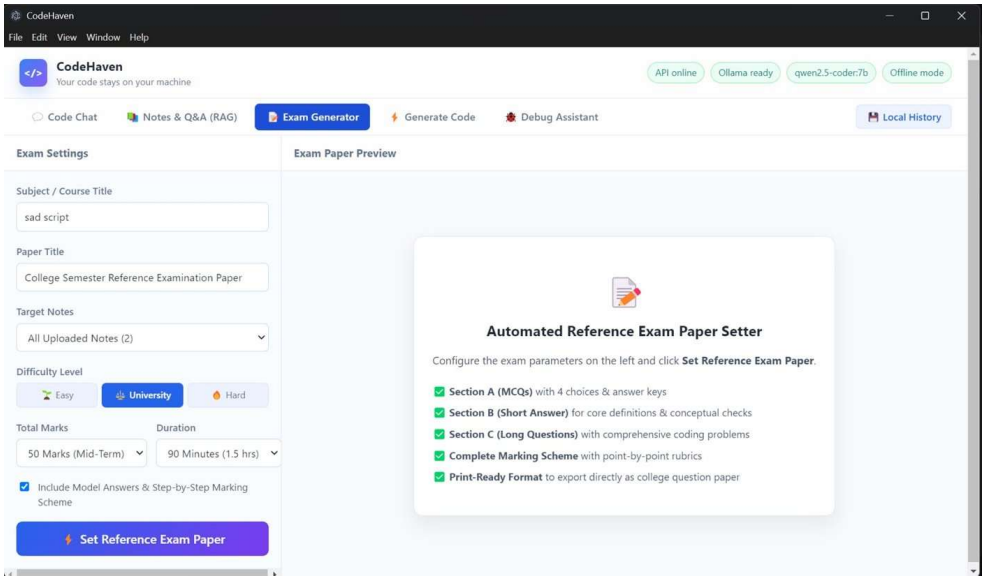


Figure 7: Exam Generator screen of CodeHaven

7.3.1 Explanation of Figure 7

The Exam Settings panel contains the fields subject / course title, paper title, target notes (All Uploaded Notes), difficulty level (Easy, University, Hard; University is selected), total marks (50 Marks, Mid-Term), duration (90 minutes), a checkbox to include model answers and a step-by-step marking scheme, and the Set Reference Exam Paper button. The preview area describes the automated reference exam paper setter and lists the parts it produces: Section A (MCQs) with four choices and answer keys, Section B (Short Answer), Section C (Long Questions), a complete marking scheme and a print-ready format.

7.4 Generate Code

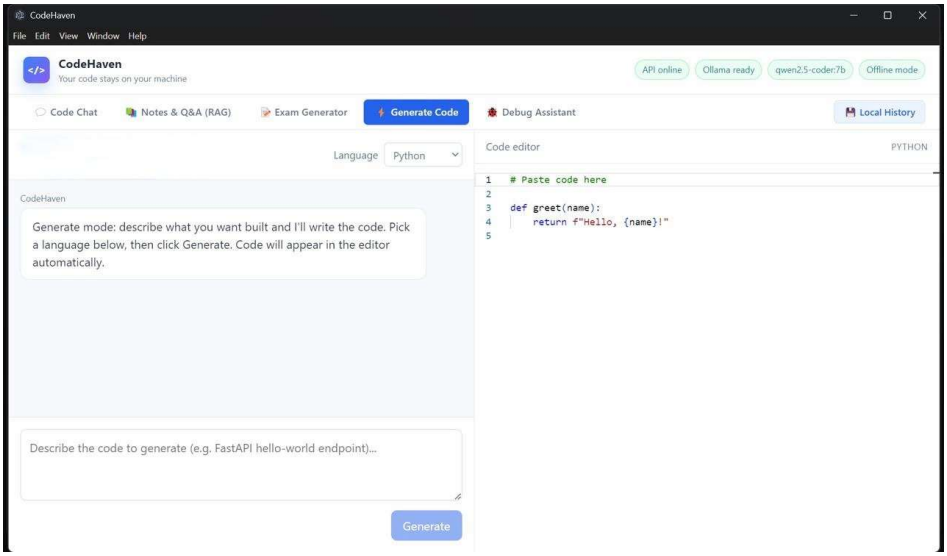


Figure 8: Generate Code screen of CodeHaven

7.4.1 Explanation of Figure 8

The chat panel explains generate mode: describe what you want built, pick a language, then click Generate, and the code appears in the editor. The language list shows Python, the editor on the right shows a sample Python function, and the input box asks the student to describe the code to generate (for example a FastAPI hello-world endpoint).

7.5 Debug Assistant

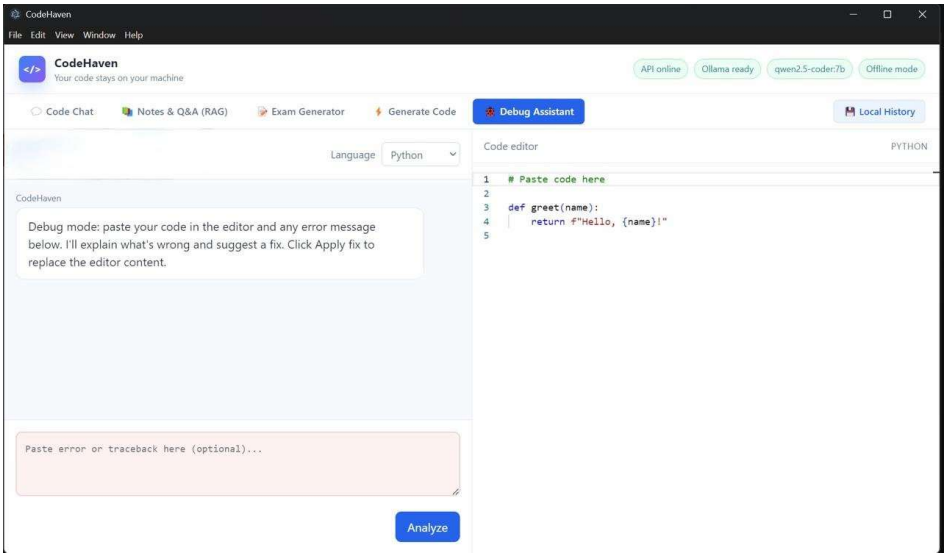


Figure 9: Debug Assistant screen of CodeHaven

7.5.1 Explanation of Figure 9

The chat panel explains debug mode: paste the code in the editor and any error message below, then the assistant explains what is wrong and suggests a fix, and Apply fix replaces the editor content. The lower box accepts an optional error or traceback and the Analyze button starts the analysis.

7.6 Local Storage & History

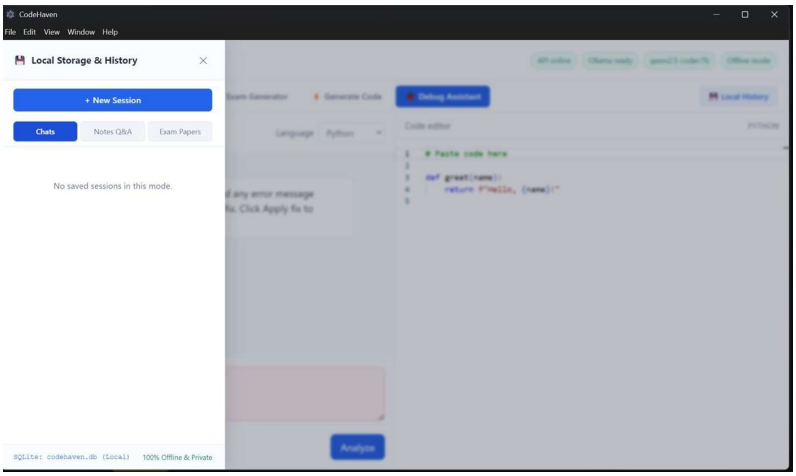


Figure 10: Local Storage & History panel, Chats tab

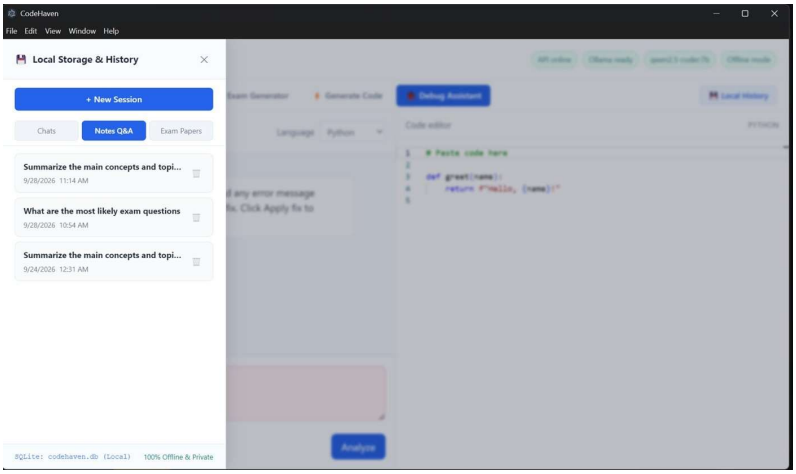


Figure 11: Local Storage & History panel, Notes Q&A tab

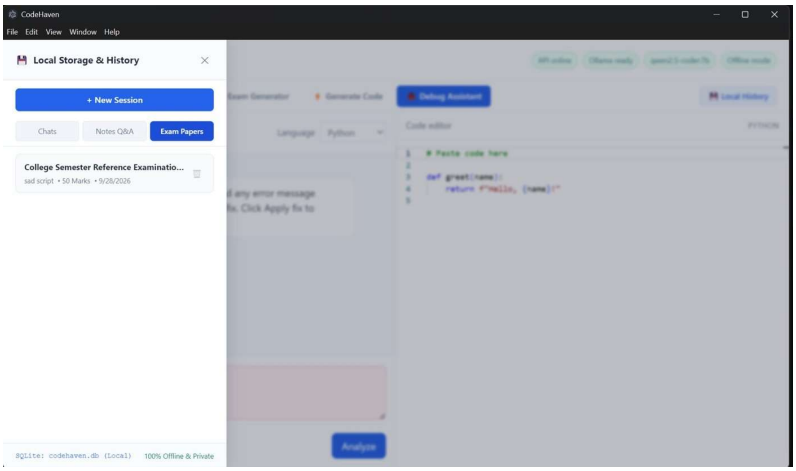


Figure 12: Local Storage & History panel, Exam Papers tab

7.6.1 Explanation of Figures 10 to 12

The Local Storage & History panel has a New Session button and three tabs: Chats, Notes Q&A and Exam Papers. In Figure 10 the Chats tab shows “No saved sessions in this mode.” Figure 11 lists three saved notes Q&A sessions with their dates and times, and Figure 12 lists one saved exam paper (“College Semester Reference Examination…”, subject “sad script”, 50 Marks). The footer of the panel reads “SQLite: codehaven.db (Local)” and “100% Offline & Private”, which corresponds to the local database of the interface layer.

CHAPTER 8
TIMELINE CHART

The source documents do not record the dates on which project activities were carried out. The chart below is therefore the **proposed plan** based on the activity durations suggested in the Major Project guidelines of the logbook (12 weeks). It is not a record of completed work. The start date and the status of each activity are **[to be filled by the team]**.

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Problem Identification and Finalization												
Proposed System and Requirement Analysis												
Literature Survey												
Project Modelling												
Project Development												
Testing and Validation												
Documentation and Report Preparation												

Figure 13: Proposed 12-week project timeline (planned, not completed work)

CHAPTER 9
FUTURE SCOPE

The research paper identifies the directions in Sections 9.1 to 9.4 for further work. Section 9.5 describes a further direction planned by the group.

9.1 Routing between Local and Cloud Models

Nothing in the framework rules out a hybrid setup in which most requests are handled locally and only hard tasks, or ones the user explicitly chooses, go to a cloud API. The authors plan to route requests between local and cloud models based on how hard the task is, keeping privacy and offline use as the default.

9.2 Lower-bit Quantization and Pruning

Lower-bit quantization and pruning are to be tested so that the working example runs well on 8 GB machines.

9.3 Benchmark across Hardware Tiers

A small benchmark is planned that compares local assistants across hardware tiers, including response speed measured on student laptops.

9.4 Larger Models for Better Hardware

Because the hardware sets the limit, a 13B model needs 16 GB of RAM or a GPU to offload layers to. The working example defaults to the smaller model and leaves larger ones as an option for students with better hardware.

9.5 Terminal-Based Local Inference Engine (Lumen Direction)

A further direction, associated with Himanshu Mishra's Lumen project, is a terminal-based inference engine for running large language models locally. Where it fits with CoreLLM, future development may involve a terminal interface, local model loading, tokenization, transformer execution, key-value (KV) cache management, sampling and inference optimisation. This would extend the study of the inference engine layer (Section 4.2.3), which this report covers through an existing engine (Ollama on llama.cpp), towards understanding how such an engine works internally.

These are planned directions only. None of these features has been implemented or tested as part of this report, and no results are claimed for them.

CHAPTER 10
CONCLUSION

This report presented a five-layer framework for running large language models entirely on local hardware – hardware, model, inference engine, augmentation, and interface and orchestration – and used the CodeHaven offline coding assistant as a working example. The example runs a 4-bit 7B model on a 16 GB laptop without a GPU, answers questions about a student's own notes through local retrieval, and accepts speech input, with no step using the network.

Local execution is not a full replacement for cloud LLMs, but it fits well when privacy, offline use or cost matter and the task is well defined. The same choices that remove the network dependency and keep data on the device also limit the assistant to what a 7B model can do.

10.1 Achievement of Objectives

Objective (Section 1.3.1)	Outcome
Explain the five-layer framework	Covered in Chapters 3 and 4 with Figure 1 and Table 3.1
Show why memory limits make quantization a requirement	Covered in Section 4.2.1 and Table 4.1 (about 14 GB at 16-bit versus about 4–5 GB at 4-bit for a 7B model)
Describe the model layer and quantization levels	Covered in Sections 4.2.2 and 4.3 with Table 4.1 and Figures 2 and 3
Explain the inference engine, key-value cache and context length	Covered in Section 4.2.3
Explain local retrieval-augmented generation	Covered in Sections 4.2.4 and 4.4
Describe the interface and orchestration layer	Covered in Sections 4.2.5, 4.6 and Chapter 7
Discuss trade-offs	Covered in Section 10.3 and Chapter 9
Measured performance on student laptops	Not yet done; identified as future work (Section 9.3)

10.2 Key Learnings

- **Every layer can be filled with existing tools.** Every layer of the framework could be filled with tools that already exist.
- **A narrow domain suits a 7B model.** Coding help for a known set of languages suits what a quantized 7B model can do.

- **The bottom of the stack limits everything above.** Memory decides which model can run, and the context length trades off against free memory.
- **Retrieval and local storage keep data on the device.** The documents, the index and the history stay local, so no network dependency is added.

10.3 Limitations and Trade-Offs

- **Capability versus privacy.** Quantized local models in the 3–13B range are not as good as the largest cloud models at open-ended reasoning. For narrow tasks with a clear output, such as completing code, explaining an error or answering questions about a given document, they are often good enough.
- **Hardware accessibility.** Because the hardware sets the limit, not every student gets the same experience. A 4-bit 7B model runs on most current laptops, while a 13B model really needs 16 GB of RAM or a GPU.
- **No speed measurements yet.** Response speed on student laptops has not yet been measured; it is planned future work.
- **Scope of evidence.** The benchmark scores reported are published scores from [10], not measurements by the project team.

10.4 Social and Educational Impact

The motivation for the work came from students: the tools need the internet all the time, the free tiers hit rate limits and people are not comfortable sending their own code to an outside server. A framework that keeps the model, the documents and the history on the student's own laptop addresses these three concerns without a cloud account.

10.5 Final Remarks

The project provides a clear, source-based foundation for understanding how a large language model inference engine fits into a complete local system. The CoreLLM-specific implementation and test results will be added by the team as they become available.

REFERENCES

[1] A. Vaswani et al., “Attention is all you need,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2017.

[2] H. Touvron et al., “LLaMA: Open and efficient foundation language models,” arXiv:2302.13971, 2023.

[3] J. Xu et al., “On-device language models: A comprehensive review,” arXiv:2409.00088, 2024.

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in Proc. Int. Conf. Learning Representations (ICLR), 2023.

[5] J. Lin et al., “AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration,” in Proc. Machine Learning and Systems (MLSys), 2024.

[6] G. Gerganov, “llama.cpp: LLM inference in C/C++,” GitHub repository, 2023. [Online]. Available: https://github.com/ggerganov/llama.cpp

[7] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2020.

[8] A. Radford et al., “Robust speech recognition via large-scale weak supervision,” arXiv:2212.04356, 2022.

[9] Ollama Inc., “Ollama: Get up and running with large language models locally,” 2024. [Online]. Available: https://ollama.com

[10] B. Hui et al., “Qwen2.5-Coder technical report,” arXiv:2409.12186, 2024.

Source paper: “A Foundational Approach to Local Large Language Model Execution”, Information Technology, M. H. Saboo Siddik College of Engineering, Mumbai.